

컴퓨터비전 개론 4주차 요약본

16. CNN 모델 빌드

16.1 CNN 등장 배경

- 기존 MLP(다층 퍼셉트론)의 한계를 해결하기 위해 CNN이 등장
- CNN은 “이미지를 위한 신경망”으로 설계됨

[MLP의 한계]

- 이미지처럼 공간적 구조($H \times W$)를 가진 데이터를 잘 활용하지 못함
- 모든 픽셀이 전부 연결되므로 파라미터 과다 → 학습 비용 증가 / 과적합 위험 증가

[CNN의 해결 방식]

1. 지역 연결(Local Connectivity)

- 작은 영역(예: 3×3 , 5×5)만 보고 특징 추출

2. 파라미터 공유(Parameter Sharing)

- 같은 필터(커널)를 전체 이미지에 적용 → 파라미터 획기적으로 감소

3. 계층적 특징 학습(Spatial Hierarchy)

- 초기 층: 에지, 선, 색감
- 깊은 층: 물체 윤곽, 형태

16.2 합성곱 연산(Convolution) 핵심 개념

- 이미지 위에 필터(커널)를 슬라이드하며 연산
- 필터와 이미지 영역의 내적(dot product) 결과가 새로운 픽셀값 생성
- 연산 결과가 모여 Feature Map(특징맵) 생성
- 필터 하나는 하나의 “특징”을 잡아내는 역할 (예: 에지 필터, 블러 필터, 색 대비 등)

16.3 CNN의 구성 요소

- Filter, Stride, Padding

요소	설명
Filter(=Kernel)	특징을 추출하는 작은 행렬 (예: 3×3, 5×5)
Stride	필터가 이동하는 간격 (1 → 촘촘하게, 2 → 더 크게 이동)
Padding	이미지 경계를 보존하기 위해 가장자리에 0을 채움

→ Padding='same' → 입력과 출력의 크기를 동일하게 유지

→ Stride↑ → 출력 이미지 크기↓ (계산량 감소)

- **Feature Map (특징맵)**

- 커널이 감지한 패턴(선·모서리·색 변화 등)을 담아낸 출력
- CNN이 학습함에 따라 필터가 점점 더 복잡한 패턴을 학습
- **중간층(Conv Layer)에서 무엇을 정확히 보고 있는지 나타내는 결과**

- **활성화 함수 (Activation Function)**

- CNN에서 가장 많이 쓰는 함수: **ReLU**
- 음수 → 0
- 양수 → 그대로

비선형성(Non-Linearity) 부여 → "딥러닝"의 핵심 조건

- **Pooling**

- **정의:** 합성곱으로 얻은 특징맵을 **축소**하는 과정

풀링 종류	역할
Max Pooling	가장 큰 값을 선택 → 가장 강한 특징만 남김
Average Pooling	평균 값으로 축소
Global Average Pooling (GAP)	특징맵 전체를 하나의 스칼라로 축소

- **목적**

- 공간 크기 감소
- 연산량 감소
- 위치 변화에 강인한 모델(translation invariance) 형성

16.4 CNN 전체 구조 흐름

입력 이미지



[Feature Extractor]

- Conv → ReLU → Pooling 반복
- 점점 더 추상적인 특징 학습



[Classifier]

- Flatten
- FC Layer (Linear)
- Softmax



클래스 확률 출력

PyTorch로 CNN 모델 기본 구성 구현 예시

특징 추출부

```
self.features = nn.Sequential(  
    nn.Conv2d(3, 32, 3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2),  
    nn.Conv2d(32, 64, 3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2)  
)
```

분류기

```
self.classifier = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(64*8*8, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10)  
)
```

• CNN 하이퍼파라미터 설계 포인트

- 필터 수 증가 규칙: 32 → 64 → 128 → 256 (깊어질수록 특징 수 증가)
- 필터 크기: 대부분 3×3
- Stride / Padding: 보통 stride=1, padding=1

- Pooling은 2×2 stride 2
- Dropout 여부: 과적합 방지에 필수 (특히 FC층)

16.5 CNN 학습 과정

1. 배치 단위로 데이터 불러오기
2. 모델에 넣어 예측(pred) 생성
3. 손실(loss) 계산 (CrossEntropyLoss)
4. 역전파(backpropagation)
5. optimizer.step()로 파라미터 업데이트
6. 반복(epoch)

[Train Mode / Eval Mode 구분]

- model.train() → Dropout 활성화
- model.eval() → BatchNorm/Dropout 비활성 + no_grad()

16.6 핵심 포인트 정리

- **CNN 핵심 구조**
 - Conv → ReLU → Pool → FC
- **CNN의 3대 핵심 개념**
 - 지역 연결
 - 파라미터 공유
 - 계층적 특징학습
- **기본 모듈**
 - Conv2d(필터)
 - ReLU(비선형)
 - MaxPool(크기 축소)
 - Flatten + Linear(Final Classifier)
- **아키텍처 설계 규칙**
 - 필터 수는 점진적으로 증가
 - 3×3 Conv + 2×2 Pool이 기본

- Feature Extractor + Classifier 구조 분리
- **PyTorch 구현 패턴**
 - `nn.Sequential` 로 특징추출부 구성
 - `forward()` 에서 features → classifier 순차 실행
- **평가**
 - Loss, Accuracy
 - Misclassified Sample 시각화
 - Confusion Matrix

17. 객체 검출 I (Two-Stage Detector)

17.1 컴퓨터 비전 주요 과제 (TASK) & 객체 검출의 도전 과제

[컴퓨터 비전의 주요 과제]

- **Classification**
 - 이미지 전체의 **무엇인지** 판단 (예: 고양이/개)
- **Localization**
 - 이미지 속에 **어디에 있는지** 위치(Bounding Box) 파악
- **Object Detection**
 - 이미지 안에 **여러 객체**를 찾고
 - **클래스 + 위치**를 함께 예측 (핵심!)
- **Instance Segmentation**
 - 객체의 픽셀 단위까지 정확하게 구분(Mask)

➡ 객체 검출은 Classification + Localization을 동시에 해결하는 더 어려운 문제

[객체 검출의 도전 과제]

- 이미지 속 객체의 규모(크기)가 다양함
- 위치·각도·조명 변화가 큼
- 여러 객체가 겹치거나 가려짐
- 하나의 이미지에 여러 개의 객체 등장

➡ 단순 분류보다 훨씬 복잡하고 계산량이 많은 문제

17.2 IoU (Intersection over Union)

- Bounding Box의 예측 정확도를 측정하는 가장 중요한 지표
- $\text{IoU} = (\text{예측 박스} \cap \text{정답 박스}) / (\text{예측 박스} \cup \text{정답 박스})$
 - $\text{IoU} = 1 \rightarrow$ 완벽한 예측
 - $\text{IoU} \geq 0.5$ 또는 0.7 기준으로 정답 여부 판단

➡ IoU는 모든 객체 검출 모델의 기본 평가 기준

17.3 NMS (Non-Maximum Suppression)

- 과정
 1. 박스들의 confidence score 기준 정렬
 2. 가장 높은 박스를 선택
 3. IoU가 높은(중복되는) 박스 제거
 4. 반복

➡ 중복 박스를 제거하고 가장 신뢰도 높은 박스를 남기는 후처리 기법

➡ 모든 객체 검출 모델에서 필수 단계

17.4 mAP (mean Average Precision)

- 객체 검출 성능을 평가하는 가장 중요한 지표
- $\text{AP} = \text{Precision-Recall}$ 곡선 면적
- $\text{mAP} =$ 모든 클래스에 대해 AP를 평균 낸 것

➡ COCO, Pascal VOC 등 주요 대회에서도 mAP가 공식 평가 점수

17.5 초기 객체 검출 방식 — Sliding Window

[초기 방법]

- 이미지를 촘촘하게 잘라서
- 모든 영역마다 분류기(CNN 혹은 SVM)를 실행

[문제점]

- 모든 위치·크기·비율을 전부 탐색해야 해서
- 연산량 폭발적 증가 $\rightarrow$ 매우 느림

➡ 이 한계를 해결하기 위해 "Region Proposal" 방식이 등장

17.6 Region Proposal

- 대표 기법: **Selective Search**
- 이미지에서 “물체일 가능성이 있는 영역”만 1~2천개 제안
- 모든 위치를 다 탐색하지 않아도 됨 → 효율성 ↑

➡ 이 방식 위에 Two-Stage Detector가 만들어짐

17.7 Two-Stage Detector란?

- 검출을 두 단계로 나누어 처리하는 방식
- **Stage 1 — Region Proposal**
 - “어디를 볼지” 찾는 단계 → RPN 또는 Selective Search 등
- **Stage 2 — Classification + Bounding Box Regression**
 - “무엇인지, 박스를 어떻게 보정할지” 최종 판단
 - → 높은 정확도가 장점
 - → 대표 모델: R-CNN 계열

17.8 R-CNN (2014) — Two-Stage Detector의 시작

- 파이프라인
 1. Selective Search로 region 2000개 추출
 2. 각 영역을 잘라 CNN에 넣어 특징 추출
 3. SVM으로 분류
 4. 회귀기로 bounding box 위치 보정
- 문제
 - 2000개 region마다 CNN을 매번 돌려야 해서 **매우 느림**

➡ 이후 모델 발전의 핵심 문제 포인트 제공

17.9 Fast R-CNN (2015) — ROI Pooling 혁신

- 핵심 아이디어
 1. 전체 이미지를 CNN에 **한 번만** 통과
 2. Feature Map 위에서 ROI(관심영역)만 잘라서 처리
 3. ROI Pooling으로 고정 크기(feature size)로 변환

4. FC Layer로 분류 + 박스 회귀

- 장점
 - 속도 대폭 향상
 - 성능은 R-CNN보다 더 높음
 - Feature Map 공유 → 효율성 최고

17.10 ROI Pooling 원리

- 문제
 - ROI는 크기가 제각각 → FC층에 넣으려면 입력 크기가 일정해야 함
- 해결
 - ROI 영역을 격자($k \times k$)로 나누고, 각 칸의 max값을 뽑아 고정 크기의 벡터로 변환

➡ 위치는 다르지만 크기를 고정해주는 "Pooling" 방식

➡ Fast R-CNN 성능·속도 혁신의 중심

17.11 Faster R-CNN (2015) — 완전 End-to-End Detector

- Region Proposal Network (RPN)
 - Selective Search 삭제
 - CNN 위에 "Proposal 생성 전용 신경망" 추가
 - Classifier + RPN이 하나의 CNN을 공유
 - 완전한 End-to-End 학습이 가능해짐
- 성과
 - 속도 대폭 상승
 - 정확도도 크게 향상
 - 현재까지도 Two-Stage Detector의 대표 모델

18. 객체 검출 II (One-Stage Detector)

18.1 One-Stage Detector 개요

- 객체 검출(Object Detection) 방법 중 속도를 높이기 위해 제안된 방식

- Two-Stage Detector (예: Faster R-CNN)는 Region Proposal → Classification 단계를 수행
반면, One-Stage Detector는 한 번의 연산으로 객체 검출과 분류를 동시에 수행
- 적용 분야: 실시간 감시, 자율주행, 드론 영상 분석 등 고속 검출이 필요한 환경에서 사용
- Two-Stage vs One-Stage: 패러다임 전환
 - **Two-Stage:** (1) 후보영역(Region Proposals) 생성 → (2) 후보별 분류·박스 보정
 - 전통적으로 **정확도 강점**, 속도는 상대적으로 느림
 - **One-Stage:** 이미지 **한 번만** 보고 **전 위치**에서 바로 **박스+클래스**를 예측
 - 연산 **파이프라인 단순화**로 **실시간** 애플리케이션에 적합
- 장점: 구조 단순, End to End, 속도 우수(지연 적음), 최근 기법으로 정확도 격차도 축소

18.2 SSD (Single Shot MultiBox Detector, 2016)

- 개요
 - CNN을 활용하여 한 번의 전방향 전파(Forward Pass)로 객체 위치와 클래스를 동시에 예측
 - VGG16(2014)을 백본(Backbone) 네트워크로 사용
 - 각 스케일에서 여러 개의 Bounding Box를 예측하는 Convolutional Predictors 사용
- 핵심 개념
 - **Bounding Box + Class Prediction**
 - 위치 예측: (S_x, S_y, w, h)
 - 클래스 예측: 여러 개의 객체 범주 예측
 - **Jaccard Overlap (IoU: Intersection over Union) 사용**
 - 예측된 Bounding Box와 실제 객체의 IoU를 계산하여 가장 적절한 박스 선택
 - **다양한 크기(Scales)와 비율(Aspect Ratios) 고려한 Default Box 사용**
 - 서로 다른 크기의 객체를 검출하기 위해 다양한 크기의 박스 생성
 - **속도가 빠르고 실시간 검출이 가능**
 - YOLO보다 다소 느리지만, Faster R-CNN보다 훨씬 빠름

18.3 SSD (Single Shot MultiBox Detector, 2016)

- 개요
 - 이미지를 $S \times S$ 그리드(Grid)로 나누고, 각 셀(Cell)에서 Bounding Box 및 클래스를 예측
 - 기존 객체 검출 방식(Selective Search, RPN)과 다르게, CNN 활용하여 한 번의 연산으로 검출 수행
 - 고속 객체 검출을 위해 개발됨
- 특징
 - **Convolutional Predictors 사용**
 - CNN의 합성곱 계층(Conv Layers)을 활용하여 객체의 위치와 클래스를 예측
 - **Unified Detection 방식**
 - 한 번의 Forward Pass로 Bounding Box 위치 및 객체 클래스를 동시에 예측
 - **End-to-End 학습 가능**
 - Faster R-CNN처럼 여러 단계로 나누지 않고 End-to-End 방식으로 학습 가능
 - **실시간 검출 가능**
 - YOLO는 Fast R-CNN보다 1000배 빠름, Faster R-CNN보다 100배 빠름

18.4 YOLO-3의 주요 개선 사항

- **다중 스케일 예측 (Multi-Scale Prediction)**
 - 큰 객체와 작은 객체를 효과적으로 검출하기 위해 3개의 출력 레이어 사용
- **Feature Pyramid Networks (FPN) 적용**
 - 다른 크기의 객체를 탐지하기 위해 여러 스케일에서 특징을 추출
- **Bounding Box 수 증가**
 - YOLOv2보다 더 많은 Anchor Box 사용 → 검출 정확도 증가
- **Darknet-53 백본 사용**
 - 이전 YOLOv2(2016)의 **Darknet-19**보다 깊은 CNN 네트워크 사용하여 성능 향상
- **출력층 구조:** 세 가지 출력 스케일을 사용하여 작은 객체부터 큰 객체까지 탐지 가능

- Faster R-CNN 및 SSD와 비교해도 뛰어난 성능을 보임

18.5 YOLO(You Only Look Once)

- “이미지를 단 한 번만 보며 모든 물체를 동시에 검출한다”는 아이디어로 **전역 맥락**을 활용해 일관된 예측을 지향
- **출력 텐서 관점**: 이미지를 **그리드**(및/또는 앵커)로 나누고, 각 위치(스케일)에서 **박스 좌표(x,y,w,h)**, **객체성(objectness)**, **클래스 확률**을 예측
- **YOLO 출력 텐서의 이해**
 - 입력 이미지를 그리드/피처맵 스케일로 나누고, 각 위치(및 앵커)에 대해 **[x, y, w, h, objectness, class_1,...,class_C]** 형태의 벡터를 산출
 - **objectness**는 “해당 박스에 물체가 있을 확률”, **class**는 “그 물체의 종류 확률”
 - 후처리: **IoU 기반 NMS**로 중복 박스 제거 → 최종 검출 결과 도출
- **FPN과 백본(Backbone)의 역할**
 - **FPN(Feature Pyramid Network)**: 상향/하향 경로로 **다중 해상도 특징**을 결합해 작은 물체까지 잘 잡도록 함
 - **Darknet-53(v3)**: 더 깊고 잔차 연결을 활용하는 백본으로 **표현력·학습 안정성** 강화
- **YOLO 손실 함수(3요소 결합)**
 - **Localization(좌표) 손실**: 예측 박스와 GT 박스의 위치/크기 차이를 최소화
 - **Objectness(신뢰도) 손실**: 물체 존재 확률을 올바르게 학습
 - **Classification 손실**: 클래스 확률의 정확도 향상
→ 3요소를 **가중 합**으로 동시에 최적화(정확·정밀·재현의 균형)
- **성능 평가지표**
 - **Precision(정밀도)**: 모델이 검출했다고 한 것 중 실제로 맞은 비율(=TP/(TP+FP))
 - **Recall(재현율)**: 실제 객체 중 모델이 맞게 찾아낸 비율(=TP/(TP+FN))
 - **mAP(mean Average Precision)**: 클래스별 AP를 평균낸 값으로, **IoU 임계값**(예: 0.5, 0.5:0.95)에서 계산
 - 목적에 따라 Precision·Recall의 **균형점**과 **mAP** 해석을 병행해야 함

18.6 YOLO 버전별 비교

버전	백본 네트워크	특징	속도(FPS)	정확도(mAP)
YOLOv1 (2016)	GoogLeNet	첫 YOLO 모델, 속도 빠름	45	낮음
YOLOv2 (2017)	Darknet-19	Batch Normalization 추가	67	개선됨
YOLOv3 (2018)	Darknet-53	다중 스케일 예측, FPN 사용	30-60	높음
YOLOv4 (2020)	CSPDarknet-53	데이터 증강, PANet 적용	50-80	향상됨
YOLOv5 (2021)	자체 백본	경량화, 빠른 속도	140+	매우 높음

19. Natural Language Processing (NLP)

19.1 자연어처리란 무엇인가?

- NLP는 인간의 언어(텍스트)를 컴퓨터가 이해·생성하도록 하는 기술
- 딥러닝을 활용해 문장을 분석하고 의미를 파악하며 다양한 언어적 작업을 가능하게 한다
- 주요 작업 영역
 - 토큰화(Tokenization): 문장을 단어/서브단어로 분리
 - 품사 태깅(POS Tagging): 단어의 문법적 역할 파악
 - 개체명 인식(NER): 사람·조직·장소 등 엔티티 인식
 - 감성 분석(Sentiment Analysis): 문장 감정(긍정/부정) 분류
 - 실제 응용 분야: 챗봇, 검색, 리뷰 분석, 고객 피드백 분석 등

19.2 텍스트 전처리 파이프라인 & 단어 임베딩의 이해

- 전처리 단계
 1. 토큰화 – 단어/서브워드 분리
 2. 정제(Cleaning) – 특수문자 제거, 소문자화 등
 3. 정수 인코딩 – 단어에 인덱스 부여
 4. 패딩(Padding) – 길이 맞추기
 5. 임베딩 변환 – 단어 → 벡터화

- 실제 예시

- 문장: "I loved the movie!"
 - 토큰: [I, loved, the, movie, !]
 - 정수 인덱스: [12, 45, 8, 99, 3]
 - 패딩 결과: [12, 45, 8, 99, 3, 0, 0, 0]

- 단어 임베딩 이해

- 원핫 인코딩(One_Hot): 차원↑ 비효율적, 단어 의미 관계 X
- 밀집 임베딩(Dense Embedding)
 - 단어 의미를 벡터 공간에 위치시킴
 - 비슷한 의미의 단어는 가까운 위치
 - PyTorch: `nn.Embedding(vocab_size, embedding_dim)`
 - 임베딩 차원 보통 128, 256, 300 등
- 임베딩의 주요 특성
 - 의미적 유사성 반영
 - 단어 간 거리 계산 가능
 - 다양한 다운스트림 작업에서 재사용 가능

19.3 LSTM (Long Short-Term Memory)

LSTM은 기존 RNN의 기울기 소실 문제를 해결하여 중요한 정보를 장기간 유지할 수 있도록 설계됨

- LSTM의 주요 개념

- 셀 상태(Cell State)
 - RNN의 hidden state와 비슷하지만, 장기적인 정보 유지 가능
- 게이트(Gates)
 - LSTM의 주요 요소로 정보를 추가하거나 제거하는 역할 수행
 - 입력 게이트(Input Gate): 새로운 정보를 셀 상태에 추가할지 결정
 - 망각 게이트(Forget Gate): 이전 정보를 유지할지 삭제할지 결정
 - 출력 게이트(Output Gate): 최종 출력값 결정, 어떤 정보를 다음으로 보낼지 결정

- **LSTM 활성화 함수**

- 시그모이드(σ) 함수 → 0~1 사이 값을 출력해 가중치 조정
- **tanh** 함수 → -1~1 범위의 값을 반환하여 RNN의 장기 의존성을 학습

- **LSTM의 장점**

- 장기 의존성 처리
- 문맥 기반 처리 가능
- 시계열·텍스트 등 순서가 중요한 데이터에 적합

19.4 GRU (Gated Recurrent Unit)

LSTM의 간소화 버전으로, 비슷한 퍼포먼스를 가지고 있는데 연산량이 감소

- **GRU의 주요 특징**

- LSTM과 유사한 기능 수행(기울기 소실 문제 해결)
- 망각 게이트와 입력 게이트를 하나로 합쳐 **연산량 감소**
- 적은 파라미터 수로도 높은 성능 유지 가능
- 비교적 작은 데이터 셋에서도 좋은 성능 발휘

19.5 Transformer로의 전환

- **RNN/LSTM의 한계**

- 순차 처리로 **병렬 처리 어려움**
- 긴 문장을 처리할수록 기억력 저하
- 연산 시간 길어짐

- **Transformer 장점**

- 병렬 처리 가능 → 효율적
- 장거리 의존성 처리 우수
- 더 많은 데이터로 학습 가능
- 큰 모델에도 적합

19.6 DistilBERT 개요

- **DistilBERT란?**

- BERT의 **경량화 모델**

- **파라미터 40% 감소, 속도 60% 증가**, 성능은 97% 유지
- 지식증류(Knowledge Distillation) 기법 활용

- **아키텍처 특징**

- 6개 레이어 (BERT-base는 12개)
- Hidden size 768
- vocab 30k+
- 모델 크기 약 66M

- **DistilBERT 학습 단계**

- 사전 학습(대규모 코퍼스)
- 분류 헤드 추가
- 파인튜닝
- 평가 및 배포

- **Bi-LSTM 장단점과 DistilBERT 장단점 비교**

[Bi-LSTM]

- 장점
 - 구조 단순
 - 적은 데이터에서도 학습 가능
 - 빠른 학습·추론
- 단점
 - 장거리 문맥 이해 한계
 - 최신 Transformer 대비 성능 뒤처짐

[DistilBERT]

- 장점
 - 강력한 문맥 이해
 - 대규모 데이터 학습 가능
 - 높은 정확도
 - Transfer Learning 가능

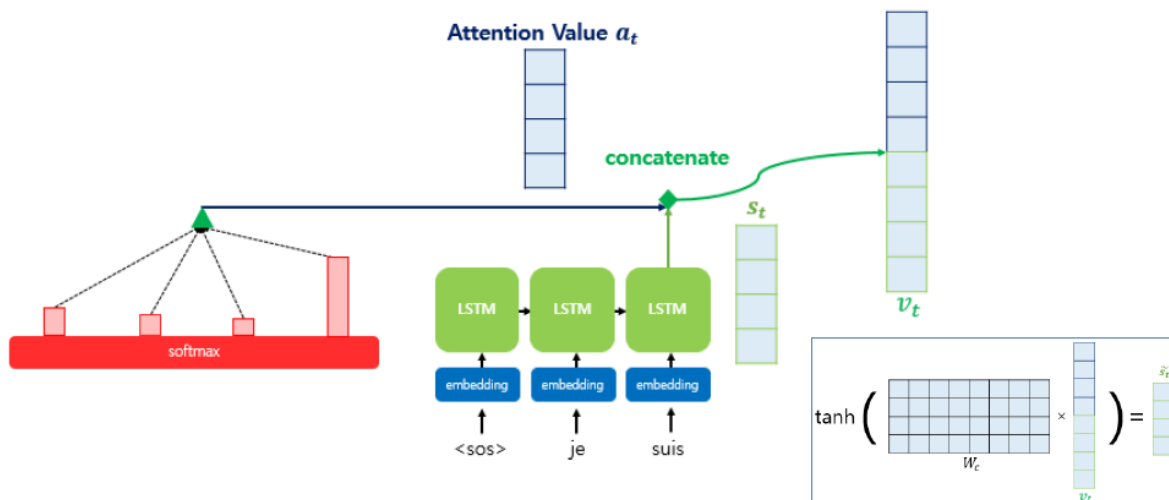
- 단점
 - 더 많은 자원 필요
 - LSTM보다 추론 느림

19.7 어텐션 (Attention)

입력의 모든 요소를 동일하게 처리하지 않고, 중요한 부분에 더 많은 가중치를 할당

- 어텐션의 필요성
 - Seq2Seq는 입력 문장을 고정된 벡터로 변환하기 때문에 긴 문장 학습이 어려움
 - Attention을 사용하면, 문장에서 중요한 단어를 더 강조하여 가중치를 다르게 적용
 - 예) "I love you"를 "나는 너를 사랑해"로 번역할 때, 각 단어가 어떻게 매핑되는지 계산 가능

▶ 어텐션 (Attention)



20. Transformer & ViT

20.1 RNN/LSTM의 근본적 한계

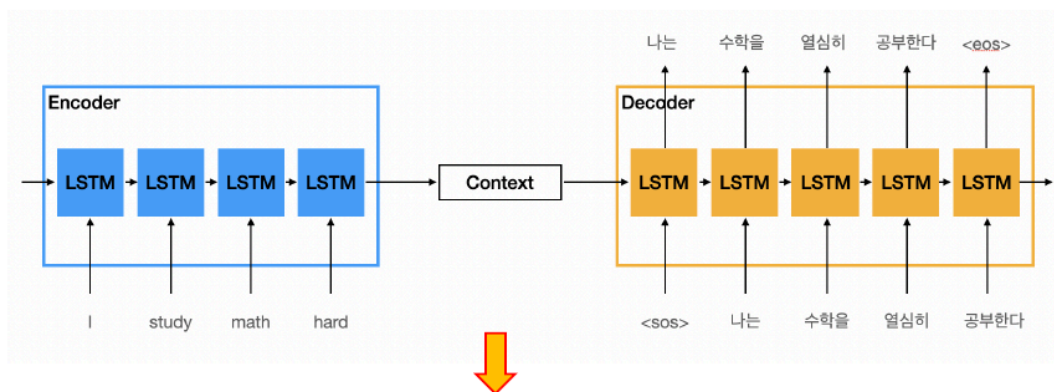
- 순차 처리 문제
 - LSTM은 입력을 앞에서부터 순차적으로 처리 → 병렬 처리 불가능
 - GPU·TPU가 가진 병렬 연산 능력을 제대로 활용하기 어려움
- 장거리 의존성 문제
 - 긴 문장일수록 **gradient vanishing/exploding** 발생

- 문장의 앞·뒤 관계를 유지하기 어려워 성능 저하
- 계산 비효율성
 - 긴 시퀀스일 때 속도가 매우 느림
 - 메모리 사용량 증가 → Out-of-memory 문제 발생 가능

➡ 이 문제를 해결하는 것이 Transformer(Self-Attention)의 등장 배경

20.2 Transformer 모델 개요

- Transformer란?
 - 기존 RNN/LSTM의 한계를 극복한 모델
 - 기존 순환 신경망(RNN, LSTM)은 장기 의존성(Long-term Dependency) 문제 有
 - Transformer는 **Self-Attention 기법**을 활용하여 병렬 연산이 가능하며, 더 빠르고 정확한 학습이 가능
 - 대표적인 Transformer 기반 모델
 - BERT (Bidirectional Encoder Representations from Transformers)
 - GPT (Generative Pre-trained Transformer)
 - ViT(Vision Transformer)
- ▶ 생성형 초거대 언어 모델 (Generative LLM)



Transformer (GPT, BERT)

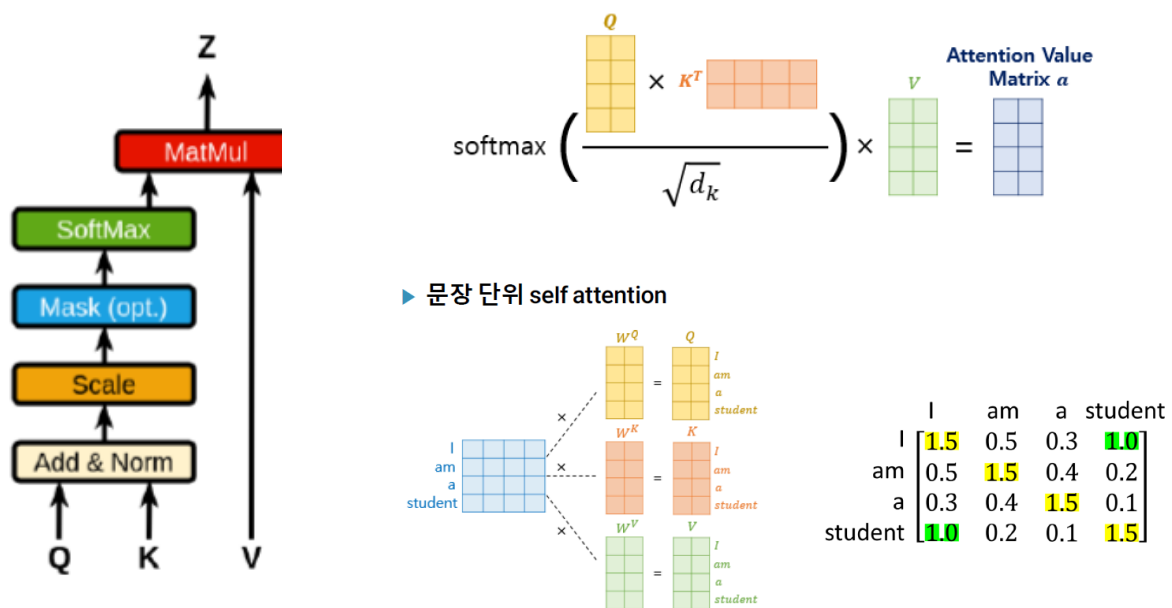
20.3 Transformer의 주요 개념

- Positional Encoding

- Transformer는 **순차적 정보**를 직접 처리하지 않기 때문에 **입력 단어의 순서를 인식할 수 없음**
- 이를 보완하기 위해 **Positional Encoding**을 사용하여 **단어의 위치 정보를 추가**
- 적용 방식: 주로 **사인(sin), 코사인(cos) 함수** 기반의 연산을 통해 위치 정보를 부여

• Self-Attention

- 입력 문장에서 각 단어가 서로 얼마나 중요한지를 학습하는 메커니즘
- 기존 RNN과 달리, 모든 단어가 서로 연관된 정보를 고려할 수 있음
- Self-Attention 연산 방식
 - **Query, Key, Value 생성**
 - **Query:** 내가 찾고 싶은 정보
 - **Key:** 단어가 가진 정체성
 - **Value:** 최종적으로 전달할 정보
 - **어텐션 가중치 계산:** Query와 Key 간 유사도를 Softmax로 변환하여 중요도를 결정
 - **Weighted Sum 계산:** Value 벡터에 어텐션 가중치를 곱하여 최종 결과 생성
 - **정리:** 각 단어의 Query가 모든 단어의 Key와 내적 → **Attention Score** 계산
→ score를 기반으로 Value를 가중합 → 문맥 반영된 새로운 표현 생성
- 장점
 - **병렬 연산 가능** → 연산 속도 향상
 - **장기 의존성 문제 해결** → 멀리 떨어진 단어 간의 관계도 반영 가능
 - **기존 RNN보다 더 정확한 문맥 이해**



20.4 Multi-Head Attention (MHA) 원리

- 왜 “Multi-Head”인가?
 - 한 가지 시각으로 문장을 이해하는 데 한계
 - 여러 개의 Head가 서로 다른 관점으로 문맥을 읽음
- 동작 과정
 - $Q/K/V$ 를 여러 Head로 분리
 - 각 Head별로 Self-Attention 수행
 - 모든 Head 결과를 Concatenate
 - 최종 W_O 를 통해 통합 출력

➡ 다양한 의미·구조적 특징을 동시에 학습할 수 있어 성능↑
- Multi-Head Attention이 학습하는 다양한 정보
 - 이미지(4개 박스)에 따르면 각 Head는 다음과 같은 관계를 학습
 - 구문적 관계: 주어-동사, 목적어-동사 등
 - 의미적 유사성: 비슷한 의미 단어끼리 주목
 - 대명사 연결: He/She/It의 참조 대상 추적
 - 장거리 의존성: 문장 끝과 처음 단어 연결

➡ 단일 Attention보다 훨씬 풍부한 문맥 이해 가능

20.5 Transformer Encoder 블록 구조

- Encoder의 구성요소

1. Embedding + Positional Encoding
2. Multi-Head Attention
3. Add & Norm (Residual + LayerNorm)
4. Feed-Forward Network (2-layer MLP)
5. Add & Norm

- 핵심 설계 철학

- Residual: 깊은 모델에서도 gradient 흐름 개선
- LayerNorm: 내부 분포 안정화
- FFN: Attention으로 모은 정보를 비선형적으로 변환해 고차원 패턴 학습
- GELU/ReLU 사용

20.6 Transformer Encoder 블록 구조

- Pre-training

- Wikipedia, BookCorpus 등
- MLM(Masked Language Model) + NSP 학습

- Fine-tuning

- 다운스트림 작업(감성 분석, QA 등)에 맞게 전체 파라미터 업데이트

- DistilBERT

- 지식증류 기반 경량 버전
- 97% 성능 유지하면서 속도 60%↑, 크기 40%↓

20.7 Vision Transformer(ViT) – 이미지를 시퀀스로 처리

- CNN 없이 순수 Transformer로 이미지 처리

- CNN의 Locality & Translation equivariance 편향 없이
- 순수 Attention 기반으로 global 정보 이해

- 이미지 패치 분할

- 224×224 이미지 $\rightarrow 16 \times 16$ 패치로 분할
- 총 $14 \times 14 = 196$ 개의 패치
- 각 패치를 flatten하여 768차원 벡터로 변환

- **Linear Projection + 위치 임베딩**

- 패치 $\rightarrow d_{\text{model}}(768)$ 로 매핑
- Positional Embedding 추가
- [CLS] 토큰 추가 가능

- Transformer Encoder 적용

- ViT-Base: 12 layers, 12 heads, hidden size 768
- 출력된 CLS embedding $\rightarrow$ MLP head $\rightarrow$ 분류

➡ 대규모 데이터(ImageNet-21K+)에서 강력한 성능

➡ 충분한 데이터가 있을 경우 CNN보다 우세